

# OO Design

# What is OO?

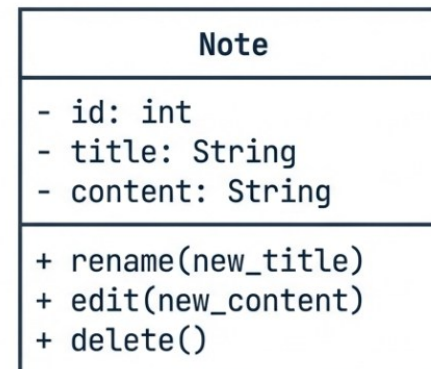
- Object-oriented (OO) design is a programming paradigm that uses "objects" to represent data and methods.
- It focuses on organizing code into reusable and modular components, promoting code reusability, maintainability, and scalability.

# Class and Objects

- Our system is decomposed into classes/objects
- A **Class** is a blueprint for an object.
  - **Attributes**: The data stored within an object represents the state of the object.
  - **Methods**: Define the behavior of an object represents what the object can do.
- An **Object** is an entity that contains both data and behavior
  - Objects are instances of classes.

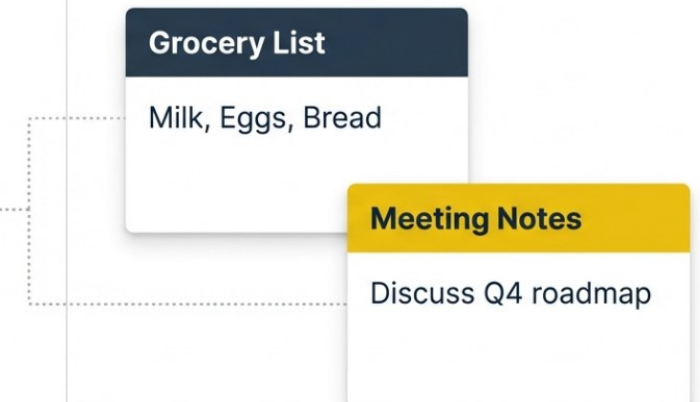
- The class name (e.g., Note) is at the top of the box.
- A minus sign (-) before an attribute or method means it is private (only accessible within the class).
- A plus sign (+) before a method means it is public (accessible from outside the class).

## THE BLUEPRINT (Class)



UML Class diagram

## THE ENTITY (Object)



# Encapsulation

- The ability of an object to hide parts of its state and behaviors from other objects
- Objects should communicate with each other via their public method, but do not need to know about how the method is implemented
- Changes to internal implementation don't break external code.
- Enforcing integrity



Rule: The notes' title should be less than 20 characters.

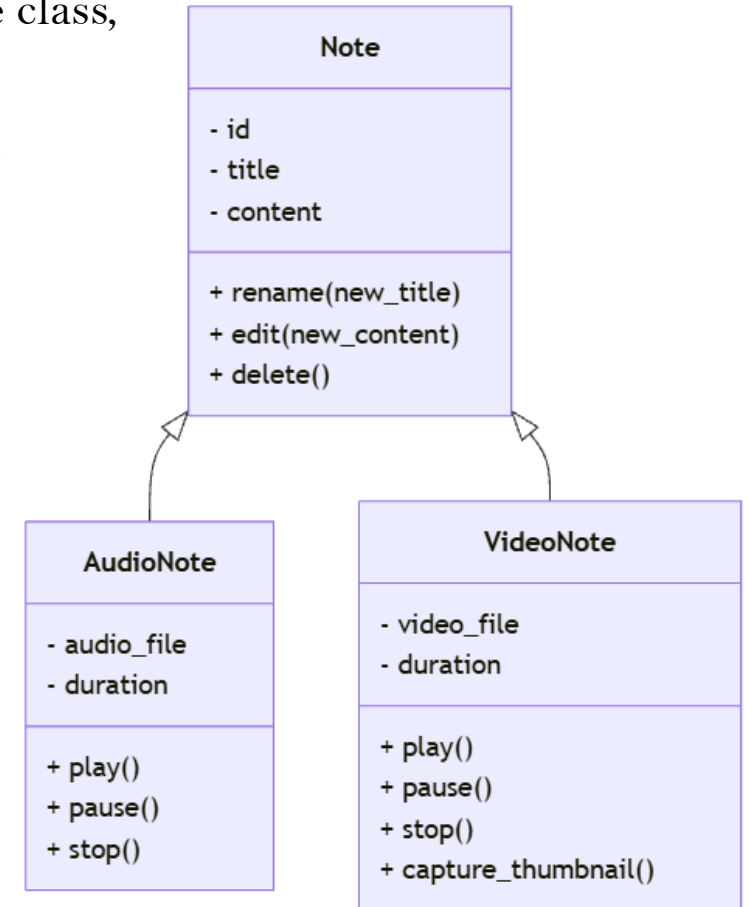
- Keep attributes private to prevent direct access from outside the class
- Use public method to interact with private attributes to enforce validation rules and maintain consistency

# Inheritance

- Let's say we want to support audio and video notes.
  - These notes will share common attributes and methods with the base Note class, but will also have their own specific attributes and methods.
- Code reuse: The subclasses inherit fields and methods of the superclass.

Inheritance is represented by a solid line with a closed, unfilled arrowhead pointing from the subclass to the superclass.

```
public class AudioNote extends Note {  
    private String audioFile;  
    private int duration;  
  
    public AudioNote(int id, String title, String content, String audioFile, int duration) {  
        super(id, title, content);  
        this.audioFile = audioFile;  
        this.duration = duration;  
    }  
  
    public void play() {  
        // Play audio logic  
    }  
  
    public void pause() {  
        // Pause audio logic  
    }  
  
    public void stop() {  
        // Stop audio logic  
    }  
}
```



UML Class diagram

# What is an Interface in OO?

- A contract between itself and any class that implements it.

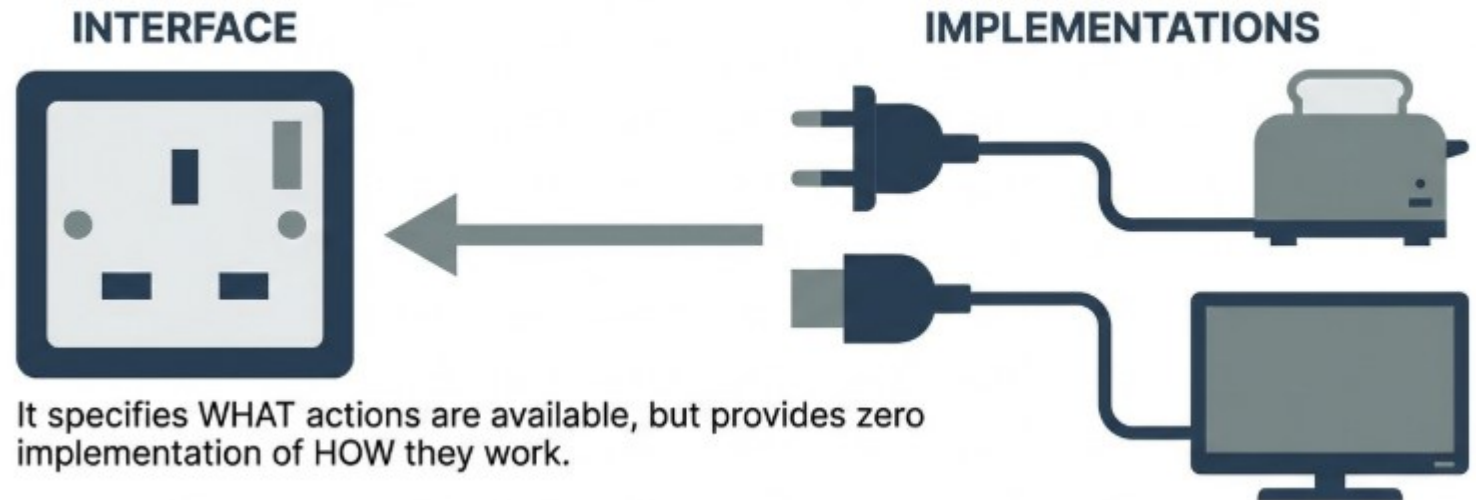
Implementation  
(source of electrical energy)



**Interface (Power outlet)**

**Client (Electrical Appliance)**

Just as a wall socket provides power without caring if it's connected to a toaster or a TV, a software Interface defines a contract.



It specifies WHAT actions are available, but provides zero implementation of HOW they work.

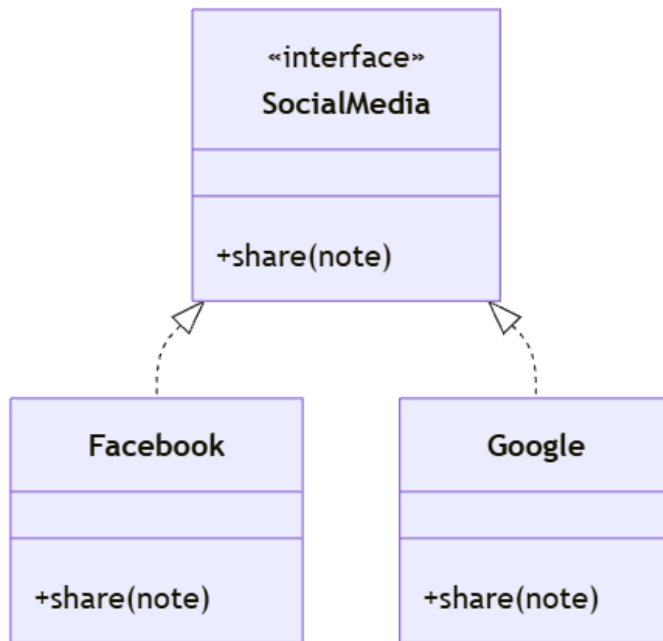
Decouples the provider from the consumer.

# Note Sharing

- In a note-taking application, users may want to share their notes on different social media platforms, such as Facebook or Google.
- We want to design this feature in a way that allows easy addition of new social media platforms in the future.

# The SocialShare interface

- An interface can provide no implementation at all.
  - The interface defines a contract (method names, number of parameters, and so on) where the developers are required to comply with the rules
  - Only the public methods are considered the interface.
    - Includes the parameters and their types, type of return values, etc.



In UML, the implements relationship (realization) is represented by a dashed line with a closed, unfilled arrowhead pointing from the implementing class to the interface.

We define a `SocialShare` interface with a method `share(note)`. Each social media platform (e.g., Facebook, Google) can then provide its own implementation of this interface.

```
public interface SocialMedia {
    void share(Note note);
}

public class Facebook implements SocialMedia {
    @Override
    public void share(Note note) {
        // Implementation for sharing note on Facebook
    }
}

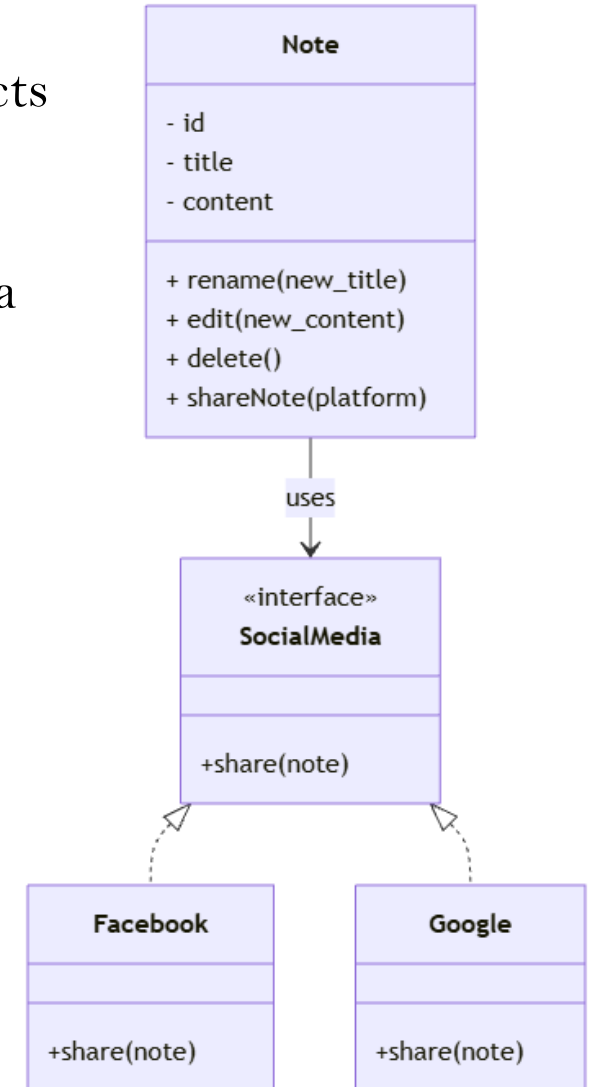
public class Google implements SocialMedia {
    @Override
    public void share(Note note) {
        // Implementation for sharing note on Google
    }
}
```



# Association (Uses-a)

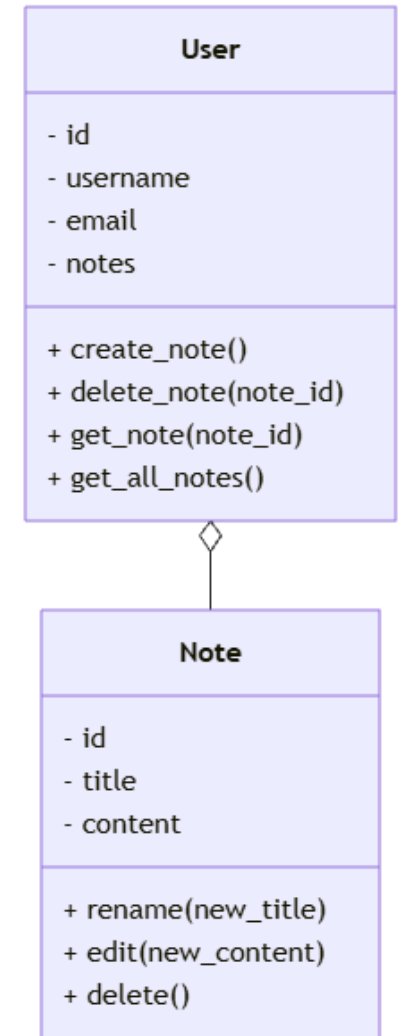
- An **association** indicates a (**Use-a**) relationship where one class uses or interacts with another class without owning it.
  - Represented by a solid line (e.g., Note uses SocialMedia)
- To allow the notes to be shared on different platforms, the Note class can have a method that accepts a SocialMedia interface

```
public class Note {  
    private int id;  
    private String title;  
    private String content;  
  
    public void shareNote(SocialMedia platform) {  
        platform.share(this);  
    }  
}
```



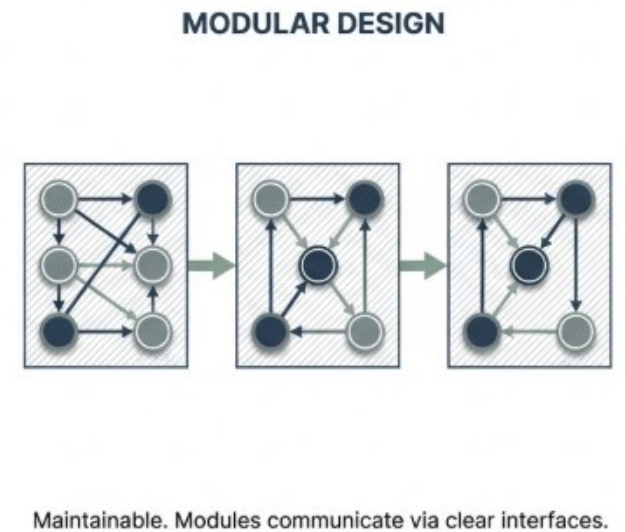
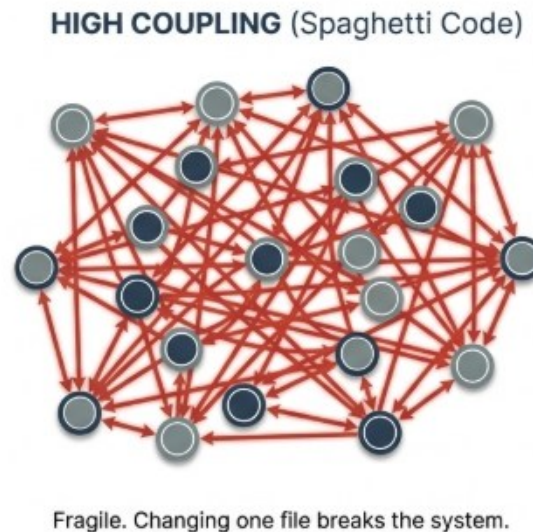
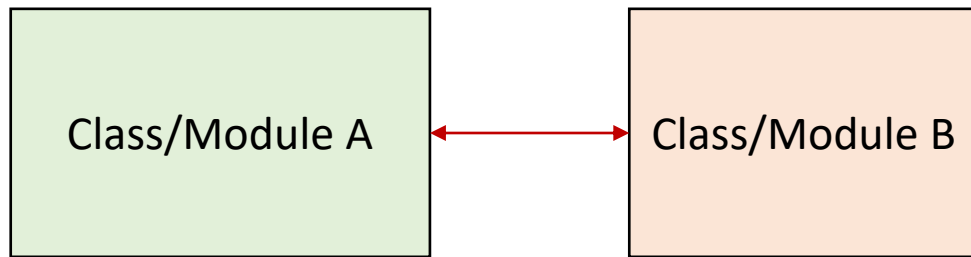
# Aggregation (Has-A)

- In OO, aggregation represents a "**has-a**" relationship where one class contains references to other classes but does not own their lifecycle.
  - For example, a User can have multiple Notes, but the Notes can exist independently of the User.
- The aggregation relationship is represented by a solid line with an open diamond at the owning class end.



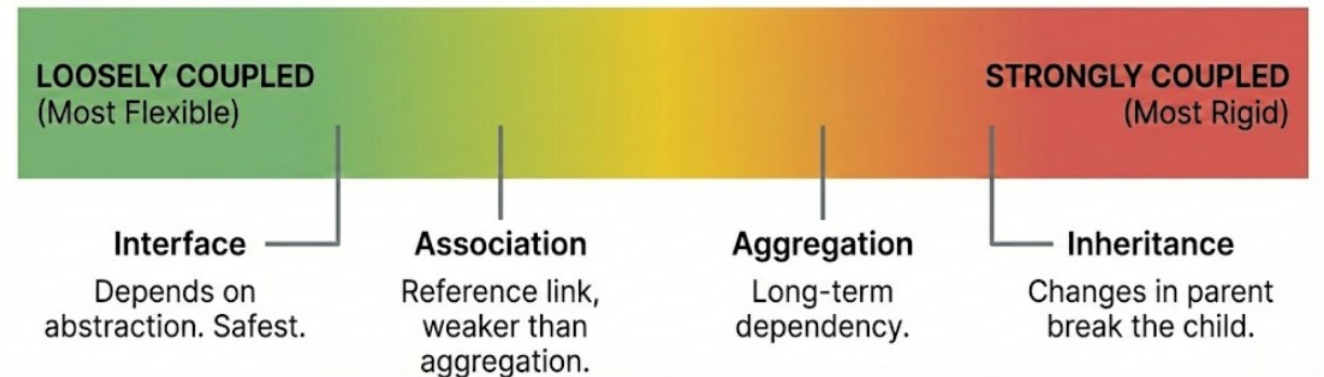
# Coupling and Cohesion in OO Design

- **Coupling** refers to the degree of interdependence between software modules or components
- **Cohesion** measures how strongly related and focused the responsibilities of a module are.
- Good OO design should be *loosely coupled* and *highly cohesive*
  - Easier to develop
  - Easier to maintain
  - Easier to add new features
  - Less fragile.



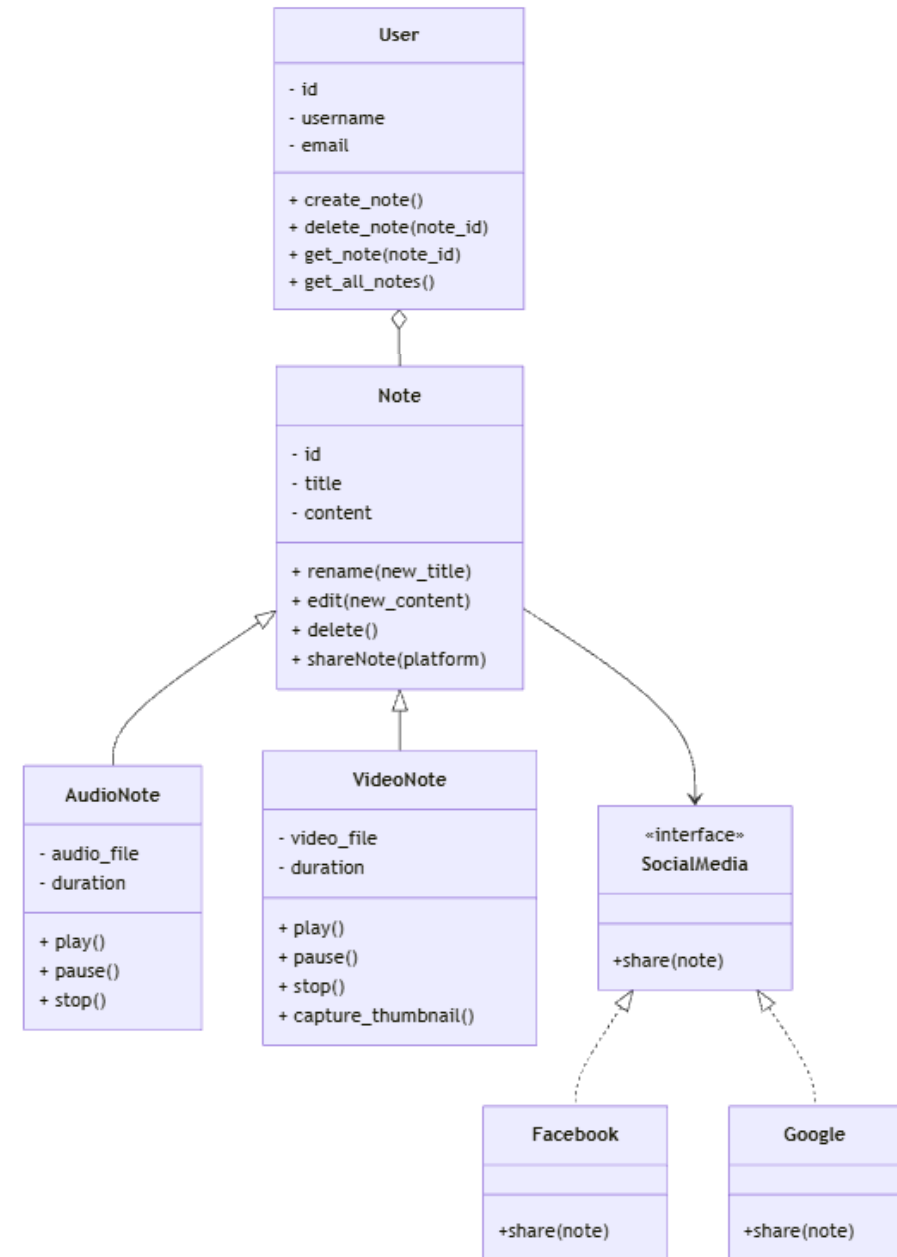
# Dependency and coupling

- In Object-oriented (OO), dependency refers to the relationships and interactions between classes and objects in an object-oriented design.
- It describes how one class relies on another class to function properly.
- Dependencies can influence how changes in one class affect other classes, impacting the overall maintainability and flexibility of the system.
- Types of dependencies in OO
  - Interface, Association, Aggregation, Inheritance
  - From most loosely coupled to most strongly decoupled



# Relationships in Notetaking App

- **Inheritance (is-a)** creates a strong coupling between the subclass and superclass
  - A child class is highly dependent on its parent class.
  - Changes in the parent class directly affect all its subclasses.
  - Subclasses inherit the implementation of the parent class=> tightly bound to the parent's internal workings.
- **Aggregation (has-a)** represents a relationship where one class contains references to other classes but does not own their lifecycle, resulting in moderate coupling.
- **Association (Use-a)** represents a weaker relationship where one class uses another class without owning it
  - Aggregation involves a long-term reference, while association is a temporary relationship
- **Interface Implementation (behaves like)** allows classes to depend on abstractions rather than concrete implementations, leading to the lowest coupling and promoting flexibility and scalability in the design.



# Design Principle: Programming to an interface, not an implementation

- A design principle that emphasizes writing code that depends on abstract interfaces or base classes rather than concrete implementations.
  - Only the public methods are considered the interface (include the parameters and their types, type of return values, etc.)
- Advantages:
  - Flexibility: Easily swap implementations without changing client code.
  - Maintainability: Changes to implementations don't affect code using the interface.
  - Clients of the class will not be affected by the implementation change

## BAD DESIGN (Implementation Bound)

```
public void shareNote(Facebook fb) {  
    ...  
}
```



Hardcoded to a specific vendor. Adding 'Twitter' requires rewriting this class.

## GOOD DESIGN (Interface Bound)

```
public void shareNote(SocialMedia platform) {  
    ...  
}
```



Agnostic to the vendor. Works with Facebook, Twitter, or LinkedIn without code changes.

# SOLID Principle

- Five design principles in object-oriented programming aimed at making software designs more understandable, flexible, and maintainable.
- Guide us from writing code that “just works” to an engineering system that lasts.
  
- SRP—**S**ingle Responsibility Principle
- OCP—**O**pen/**C**lose Principle
- LSP—**L**iskov Substitution Principle
- IPS—**I**nterface Segregation Principle
- DIP—**D**ependency Inversion Principle

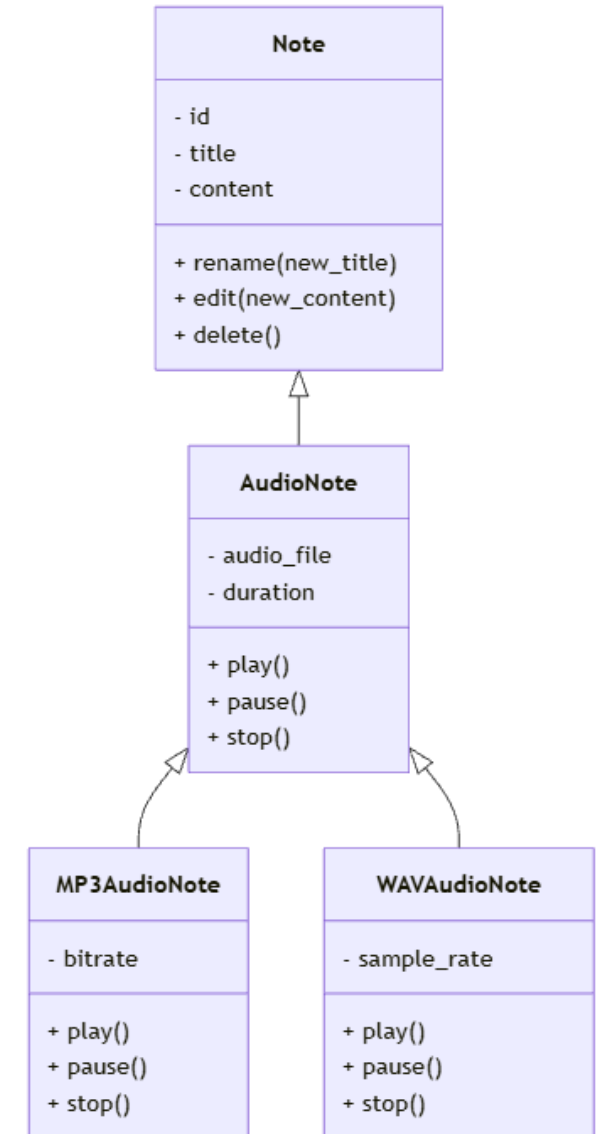
# Open-closed principle

- Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification
- You should be able to extend a class's behavior without modifying it.



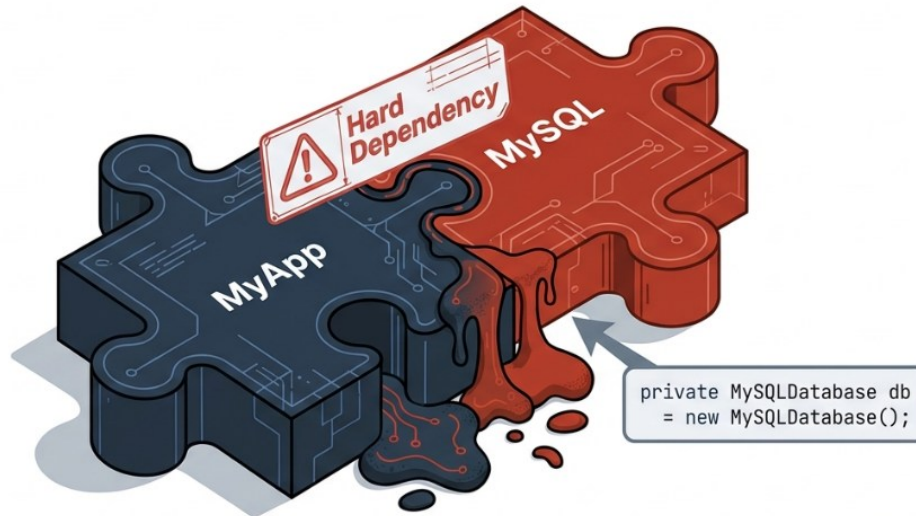
# Example: Extending AudioNote

- **Polymorphism:** Objects of different classes can be treated as objects of a common superclass or interface.
  - A subclass is-A superclass:
  - Any code that works with Note objects can also work with AudioNote objects without modification.
- Suppose that you want to extend the AudioNote class to support different audio formats, such as MP3 and WAV.
- Create subclasses for each format.
  - A player that works with AudioNote objects can also work with MP3AudioNote and WAVAudioNote objects without any changes
  - This adheres to the Open/Closed Principle.



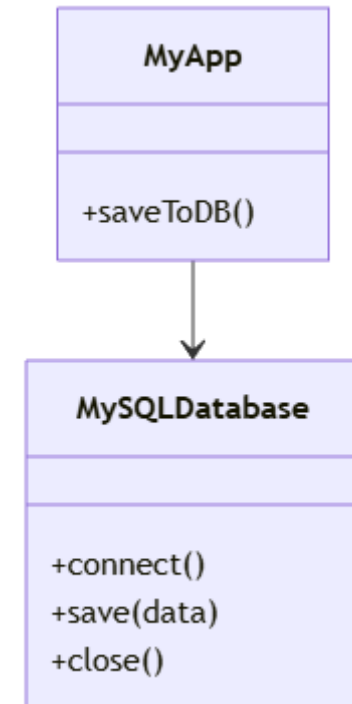
# Example: Databases

- **MyApp** is tightly coupled to **MySQLDatabase** (Concrete class)
- If we want to change the database type (e.g., from **MySQL** to **PostgreSQL** or **MongoDB**), we would need to modify the **MyApp** class.

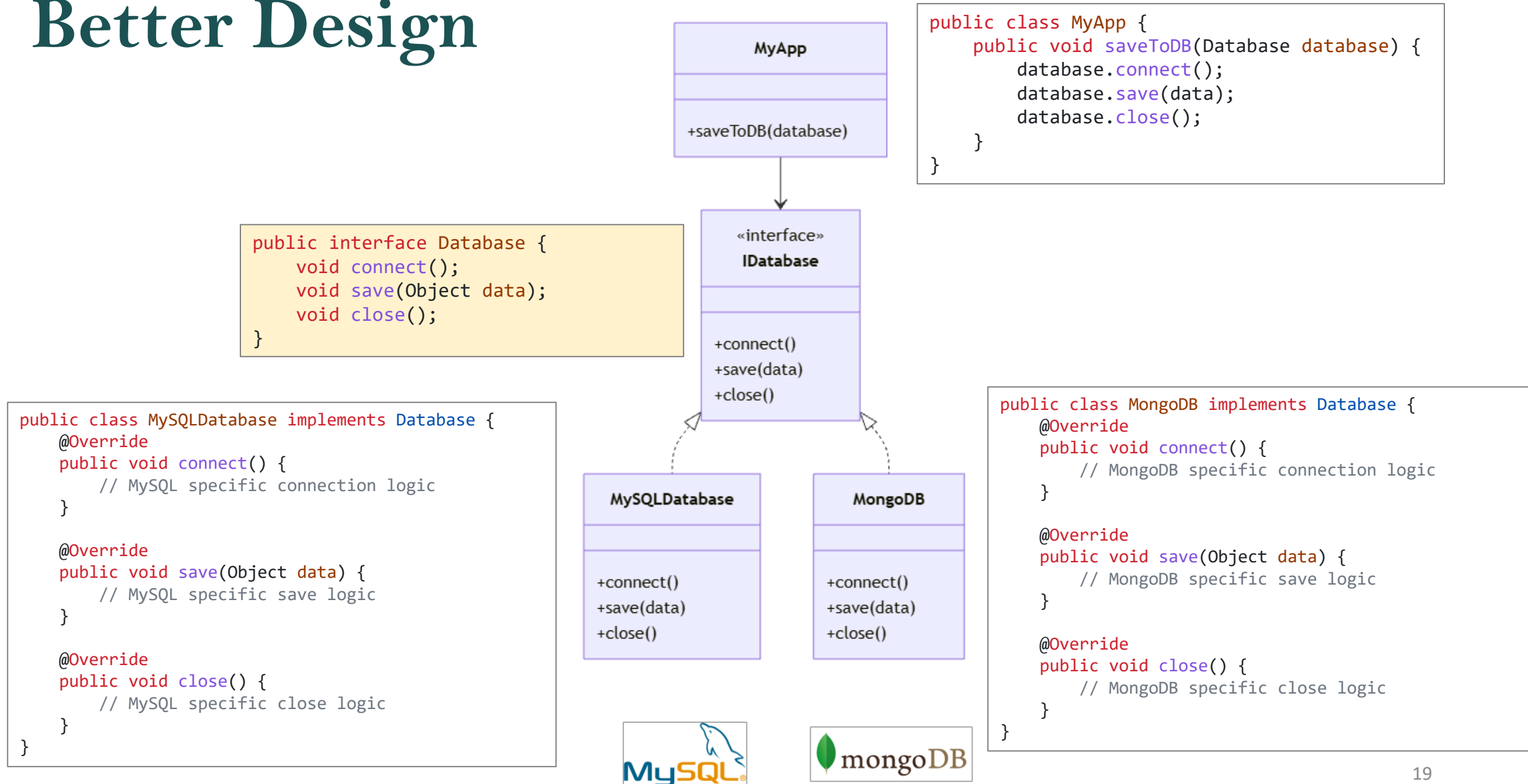


By instantiating 'MySQL' directly inside the App, we are tightly coupled. To switch to MongoDB, we must break open and modify the App code. This violates the principle: Open for extension, Closed for modification.

```
public class MyApp {  
    public void saveToDB() {  
        MySQLDatabase db = new MySQLDatabase();  
        db.connect();  
        db.save(this.data);  
        db.close();  
    }  
}
```



# Better Design



# Dependency Inversion Principle (DIP)

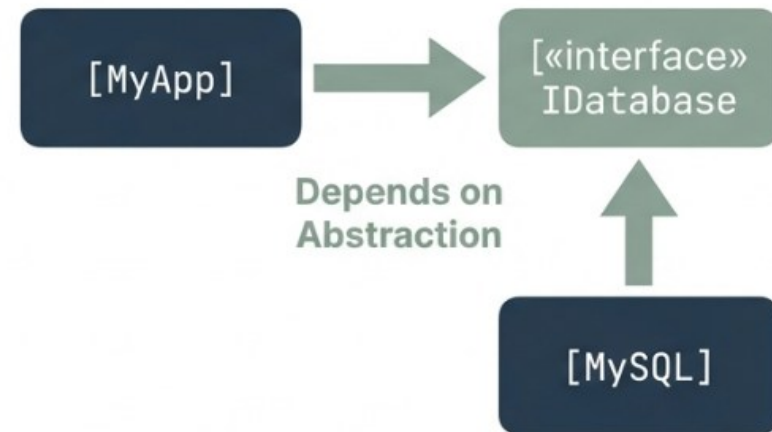
High-level modules should not depend on low-level modules. Both should depend on abstractions.

- **High-level modules**
  - Modules that contain the business logic of an application.
- **Low-level modules**
  - These are modules that handle specific operations
    - E.g. database, logging modules
- **Abstractions**
  - Interfaces or abstract classes that define contracts for functionality.

## BEFORE



## AFTER (DIP)



# Dependency Injection

- Dependency Injection (DI) is a design pattern and programming technique that aims to reduce the coupling between different parts of a software system.
  - A dependency is any external component that a module, class, or function needs to perform its tasks.
- Instead of a class creating or managing its own dependencies, these dependencies are provided to the class from an external source. Types:
  - **Constructor Injection:** Dependencies are provided through a class constructor.
  - **Setter Injection:** Dependencies are provided through setter methods.
  - **Method Injection:** Dependencies are provided through method parameters.

  
`// App creates its own dependency (Bad)  
public MyApp() {  
 this.db = new MySQLDatabase();  
}`

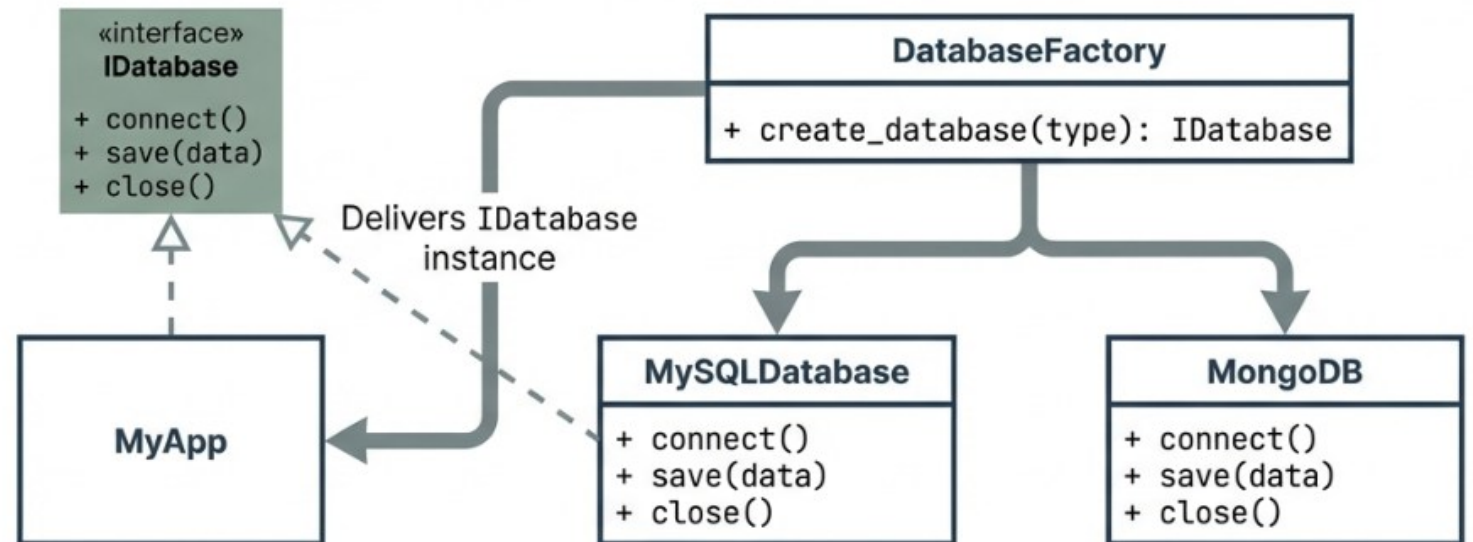


  
`// Dependency is injected from outside (Good)  
public MyApp(IDatabase db) {  
 this.db = db;  
}`

External system provides the  
dependency via the constructor.

# Factory pattern

- If the App doesn't create the database, who does?
- The “Factory” encapsulates the messy creation logic
  - Decide which database to build based on the configuration
  - Keep the main app clean



# Advantages of Dependency injection in OO development

- Allows for loose coupling between components
  - Easier to swap out implementations without changing the code that uses them.
- Enhanced Testability: By injecting dependencies, you can easily substitute mock objects or stubs (e.g. fake databases) during unit testing
- Dependency injection is a core principle in frameworks like Java Spring
  - In Spring, the Inversion of Control (IoC) container is responsible for creating and managing the lifecycle of objects
  - The dependencies are automatically injected by the container

